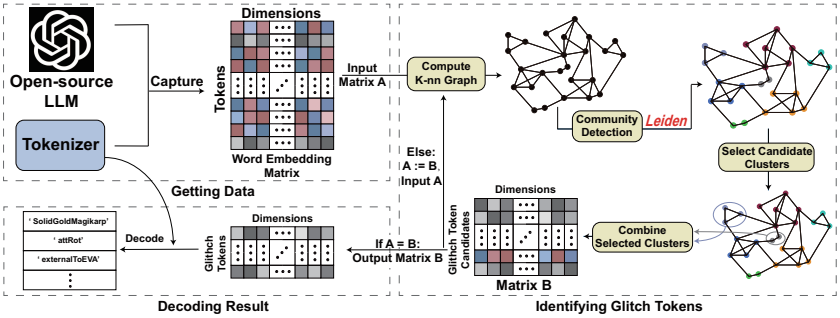

Fig. 5. Overall Workflow of GlitchHunter

4.4 Implications of Our Findings

In this section, we outline how the findings from Section 4.1 to 4.3 assist LLM developers in mitigating glitch tokens.

Test Oracle for Glitch Token. In our study, we introduce the concept of the "repetition relation" as a reliable test oracle. This method is specifically designed to discern and identify glitch tokens within LLMs. By leveraging this relation, developers can effectively pinpoint and address glitch tokens, enhancing the overall performance and reliability of LLMs.

Efficient Glitch Token Detection. Efficient detection of glitch tokens is crucial, especially as LLMs consume significant computational resources. The computational demand grows quadratically with parameter size. In our experiments, iterating through all tokens in the vocabulary dictionary typically takes over a week. Given our observation that glitch tokens cluster in the embedding space, we introduce GlitchHunter. This tool accelerates glitch token detection by strategically searching within the embedding space.

5 EFFICIENT GLITCH TOKEN DETECTION (RQ4)

Based on our empirical research, we introduce GlitchHunter, an automated method that utilizes iterative clustering to identify glitch tokens in LLMs.

Figure 5 illustrates GlitchHunter's workflow. Initially, GlitchHunter constructs the Token Embedding Graph (TEG) using all tokens and their respective embedding vectors (Section 5.1). Next, it conducts candidate clustering on the initial TEG to generate potential glitch token clusters (Section 5.2). Within each cluster, GlitchHunter conducts a hypothesis test to identify those with glitch tokens. Tokens from these selected clusters are then integrated into an updated TEG. This process concludes one iteration and GlitchHunter continues clustering until the TEG experiences no further updates.

5.1 Initial TEG Building

In GlitchHunter, we introduce the Token Embedding Graph (TEG) to represent relationships between tokens in the embedding space, drawing inspiration from the **Weighted K-nn Graph**. For a set of data points $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$, each point x_i corresponds to a vertex in V, leading to $V = \mathcal{D}$. We determine the k nearest neighbors for every data point x_i, denoted $N_k(x_i)$. An edge $e = (x_i, x_j)$ exists if x_j is in $N_k(x_i)$ or vice versa. The edge weights are provided by function W.

Given the **Weighted K-nn Graph** framework, we define the Token Embedding Graph (TEG) as follows: In TEG, each token t represents a vertex in V. An edge $e = (x_i, x_j)$ is presented if x_j belongs to the k nearest neighbors of x_i, or the other way around. We define

$$\rho_i := \min\{d(x_i, x_j) | j \in N_k(x_i), d(x_i, x_j) \geq 0\}$$

as the minimum distance in $N_k(x_i)$, and set σ_i to be the value such that

$$\sum_{j \in N_k(x_i)} exp(\frac{-\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}) = log_2(k)$$

Edge weights are determined by function W, with calculations as outlined in

$$W((x_i, x_j)) = exp(\frac{-\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i})$$

Building the TEG requires the embedding matrix from the target LLM. This matrix, denoted as $A \in \mathbb{R}^{n \times m}$, serves as the raw data. Here, n is the total number of tokens, and m is the dimension of each token.

Design Rationale. The foundation of TEG's design stems from a key finding gathered from our prior study: glitch tokens frequently cluster in proximate regions within the embedding space. In constructing the TEG, we conceptualize each token t as an individual vertex in V. To capture the inherent relationships between tokens, we leverage the embedding distance between them, which allows us to assign meaningful weights in W. Adopting the K-nn graph structure proves advantageous as it intuitively and effectively encapsulates the natural clusters formed by tokens that share closer distances in the embedding space. This structure offers a strategic advantage when identifying potential glitch tokens. Thus, we apply k-nearest-neighbor to build the TEG.

5.2 Candidate Clustering

In GlitchHunter, we aim to pinpoint glitch tokens by continuously refining token clusters. Observing that glitch tokens cluster tightly in the embedding space, we focus on these dense groupings to narrow down our search. We employ the Leiden algorithm [41], renowned for its efficiency, to amplify the glitch token density in our matrix, as outlined in Algo 1. Initially, we set matrix B to reflect the word embedding matrix A (Line 1). Using the default k value of 50 in the K-nn algorithm, we construct the TEG G and derive the weighted adjacency matrix W (Line 3). Setting the default resolution γ to 75, we input TEG G and matrix W into the Leiden algorithm (Lines 4-6). A greater γ yields more clusters. Post-Leiden processing, we sample tokens from clusters and validate them with the oracle described in Section 4.2 to identify glitch token areas (Lines 7-16). If a sampled cluster's glitch token ratio surpasses threshold r, we mark it as a glitch token source (Lines 12-14). We adjust the temperature to 0 to ensure consistent LLM outputs. Matrix C compiles aggregates from the selected clusters. We then assess the outcome of each iteration (Lines 17-21). If matrices C and B match, the process halts, considering C's concentration adequate (Lines 17-18). If C is slimmer than B, the iterations persist with $B := C$ (Lines 19-21). When GlitchHunter achieves a stable TEG without further modifications, it outputs the remaining tokens.

Leiden vs Other Clustering Algorithms. In GlitchHunter, we favor the Leiden algorithm for clustering, and our rationale is grounded in its comparison with other popular clustering methods. While k-means [17] is a widely used approach, it tends to be sensitive to the initial positioning of centroids, rendering it less stable. On the other hand, k-means++ [27] and bi-kmeans [51], though advanced, fail to optimally leverage our discovery that glitch tokens are often closely situated. DBSCAN [11], another renowned algorithm, struggles with data points exhibiting uneven densities, a characteristic we've observed in our datasets. Agglomerative Hierarchical Clustering [19], despite its robustness, is less appealing due to its high time complexity of $O(n^3)$. In contrast, the Louvain algorithm [2], which is akin to the Leiden algorithm, boasts a more efficient time complexity of $O(m \log n)$, especially relevant when $k = 50$ leading to $m = 50 \times n$. Hence, considering these factors, the Leiden algorithm stands out as the most apt choice for our glitch token detection task.

Algorithm 1: GlitchHunter

```
Input: Word embedding matrix A, Threshold r, Resolution γ, k
1  B := A;
2  while True do
3      (G, W) := K-nearest-neighbor(B, k);
4      n = B.rowNumber();
5      P := {{v}|1 ≤ v ≤ n};
6      P := Leiden(G, W, P, γ);
7      i := 1;
8      C := ∅;
9      while i ≤ |P| do
10         S_i = RandomSample(P_i) ;
11         x = GlitchTokenTest(S_i) ;
12         if x/|S_i| ≥ r then
13             C := C ∪ P_i;
14         end
15         i := i + 1;
16     end
17     if n = C.rowNumber() then
18         break;
19     else
20         B := C;
21     end
22 end
Output: target glitch token set C
```

Time Complexity Analysis. GlitchHunter demonstrates a lower time complexity than the baseline approach of enumerating each token in the vocabulary, improving glitch token detection efficiency. Intuitively, identifying glitch tokens for all proxy tasks involves checking every token, leading to a time complexity of $O(n)$, where n denotes the vocabulary size. However, GlitchHunter minimizes the time for K-nearest-neighbors calculations and Leiden algorithm executions relative to LLM inferences. The critical efficiency gain in GlitchHunter stems from reduced model inferences, as only a 0.05 fraction of tokens from each cluster is sampled for validation, as specified in Algorithm 1, line 11. While maintaining linear complexity, $O(n)$, this method significantly lowers the constant factor, thereby reducing the overall time requirement. A full efficiency analysis is detailed in Section 6.

6 EFFECTIVENESS AND EFFICIENCY OF GLITCHHUNTER

We have implemented GlitchHunter to detect glitch tokens, and we release our code and results on our website [15]. To evaluate the performance of GlitchHunter, we carry out a series of experiments. Specifically, our assessment seeks to address the following research question:

•**RQ5: (Efficiency and Effectiveness)** How efficient and effective is our approach in identifying glitch tokens in different LLMs?